CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

# UNIVERSITY INSTITUTE OF COMPUTING

# Project Report
# on
# Sales Data Summary Tool

## Subject Name: Object Oriented Programming

## Subject Code: 25CAH-201


## SUBMITTED BY:

Name: Ishika Puri

UID: 25BCD10038

Branch: BCA-Data Science

Semester: 3

Section: 1

Group: B


## SUBMITTED TO:

Name: Ms. Shuchi Sharma

Designation: Assistant Professor

Date of Submission:

05.11.2026

Signature:

# 1. ABSTRACT

The Sales Data Summary Tool is a console-based application developed using the C++ programming language. The main purpose of this project is to analyse monthly sales data stored in a CSV (Comma-Separated Values) file and generate a meaningful summary.

The application automatically calculates total revenue, total units sold, average monthly revenue and average unit price. It also identifies the month with the highest and lowest sales, calculates the difference between them, and displays months having revenue above the average.

Additionally, the system calculates month-to-month revenue growth and generates a simple text-based bar chart to represent sales performance. The final report is displayed on the console and automatically saved in a file named report.txt.

This project demonstrates the practical implementation of C++ concepts such as file handling, structures, vectors, functions, strings and standard algorithms.

# 2. INTRODUCTION AND PROBLEM STATEMENT

## 2.1 Introduction

Sales analysis plays an important role in understanding the performance and growth of a business. Businesses generate sales records regularly, and analysing this information helps identify profitable months and understand sales trends.

Manually calculating monthly sales totals, averages and comparisons can be time-consuming and may lead to calculation errors.

To overcome this problem, the Sales Data Summary Tool has been developed using C++. It reads sales records from a CSV file, processes the information and generates an organised sales report. The project provides an automated way of analysing monthly sales data without requiring an external database or additional software packages.

## 2.2 Problem Statement

The main problem addressed by this project is the difficulty of manually analysing large amounts of monthly sales data. The project aims to develop a C++ application that can:

Read monthly sales information from a CSV file.

Calculate total revenue and total units sold.

Calculate average monthly revenue and average unit price.

Identify the best-performing and worst-performing months.

Calculate monthly revenue growth.

Display sales information in a simple text-based chart.

Generate and save a complete sales summary report.

## 3. OBJECTIVES

Automated Data Reading: Read monthly sales records directly from a CSV file.

Revenue Calculation: Calculate the total revenue generated during the selected period.

Units Calculation: Calculate the total number of units sold.

Average Calculation: Determine average monthly revenue and average unit price.

Performance Analysis: Identify the month with the highest and lowest revenue.

Sales Comparison: Calculate the revenue difference between the best and worst months.

Growth Analysis: Calculate the percentage change in revenue between consecutive months.

Data Representation: Display monthly revenue through a text-based bar chart.

Report Generation: Display and save the final sales report in a text file.

Error Handling: Identify invalid CSV records and skip them without stopping the program.

## 4. SOFTWARE AND HARDWARE REQUIREMENTS

### 4.1 Software Requirements

| COMPONENT | REQUIREMENT |
|---|---|
| Programming Language | C++ |
| C++ Standard | C++17 |
| Compiler | g++ Compiler |
| Code Editor | Visual Studio Code or any text editor |
| Input File | sales.csv |
| Output File | report.txt |

### 4.2 Hardware Requirements

| COMPONENT | REQUIREMENT |
|---|---|
| Processor | Basic computer processor |
| RAM | Sufficient memory to run a C++ compiler |
| Storage | Space for source code and CSV files |
| Computer | Desktop or laptop |
| Keyboard | For writing and executing the program |

## 5. PROJECT FEATURES

### 5.1 CSV File Input

Reads monthly sales information from a CSV file containing month names, revenue and units sold.

### 5.2 Total Revenue Calculation

Calculates total revenue by adding the revenue of all valid records.

### 5.3 Total Units Sold

Calculates the total number of units sold during the selected period.

### 5.4 Average Revenue Calculation

Calculates average monthly revenue using total revenue and the number of valid records.

### 5.5 Best and Worst Month Identification

Identifies the month with the highest revenue and the month with the lowest revenue.

### 5.6 Revenue Difference

Calculates the difference between the highest and lowest monthly revenue.

### 5.7 Above-Average Sales

Identifies all months whose revenue is greater than the overall average revenue.

### 5.8 Month-to-Month Growth

Calculates the percentage increase or decrease in revenue between consecutive records.

### 5.9 Text-Based Bar Chart

Displays a simple revenue chart using the '#' character to represent monthly sales.

### 5.10 Report Generation

Displays the final analysis on the console and saves it automatically in report.txt.

### 5.11 Invalid Data Handling

Skips invalid CSV rows and displays a warning instead of terminating the program.

## 6. WORKING AND ALGORITHM

### 6.1 Working of the Project

Input File Selection: The program selects the CSV file. If no filename is provided, it uses sales.csv by default.

File Opening: The loadCSV() function opens the CSV file and skips its header row.

Data Reading: The program reads monthly records containing the month name, revenue and units sold.

Data Separation: Each record is separated into individual fields using string processing techniques.

Data Validation: The program checks whether records can be converted into valid numeric values. Invalid records are skipped with a warning.

Data Storage: Valid records are stored in a vector of Record structures.

Calculations: The program calculates total revenue, total units, average revenue, average unit price and monthly growth.

Performance Comparison: It identifies the highest-revenue and lowest-revenue months.

Report Preparation: All calculated information is arranged in a structured sales report.

Output Generation: The report is displayed on the console and saved in report.txt.

### 6.2 Algorithm

Start the program.

Select the CSV input file.

Open the CSV file and read its contents.

Skip the header row.

Read each monthly sales record.

Validate the data and skip invalid records.

Store valid records in a vector.

Calculate total revenue and total units sold.

Calculate average monthly revenue and average unit price.

Identify the best and worst sales months.

Calculate the revenue difference and monthly growth percentage.

Generate a text-based bar chart.

Display the complete report on the screen.

Save the report in report.txt.

Stop the program.

# 7. C++ CONCEPTS AND FUNCTIONS USED

## 7.1 C++ Concepts Used

| C++ CONCEPT | APPLICATION IN THE PROJECT |
|---|---|
| Structures | The Record structure groups the month name, revenue and number of units sold. |
| Vectors | vector<Record> stores a flexible number of monthly sales records. |
| Functions | Separate functions perform operations such as reading data, calculating totals and generating reports. |
| File Handling | ifstream reads information from sales.csv and ofstream writes the final output into report.txt. |

| | |
|---|---|
| Strings and StringStream | Strings store month names and CSV fields, while stringstream helps separate information from individual CSV records. |
| Standard Algorithms | max_element() and min_element() identify the months with the highest and lowest revenue. |
| Exception Handling | The try-catch mechanism handles invalid numerical values during CSV processing. |
| Command-Line Arguments | argv[1] allows users to provide a custom CSV filename while executing the program. |

### 7.2 Important Functions Used

| FUNCTION / STRUCTURE | PURPOSE |
|---|---|
| Record | Stores one month's name, revenue and units. |
| trim() | Removes unnecessary spaces and line-ending characters. |
| loadCSV() | Reads CSV data and stores valid records. |
| totalRevenue() | Calculates total sales revenue. |
| totalUnits() | Calculates total units sold. |
| bestMonth() | Finds the month with the highest revenue. |
| worstMonth() | Finds the month with the lowest revenue. |
| printBarChart() | Displays the monthly revenue bar chart. |

| | |
|---|---|
| printReport() | Generates and displays the complete sales report. |
| main() | Controls the execution of the entire program. |

## 8. COMPILATION AND EXECUTION STEPS

The following steps are used to compile and execute the Sales Data Summary Tool.

### Step 1: Install a C++ Compiler

Install the g++ compiler on the computer.

### Step 2: Create the Source File

Write the C++ program and save it as main.cpp.

### Step 3: Prepare the CSV File

Create a CSV file named sales.csv in the same folder.

### Step 4: Open the Terminal

Open the terminal or command prompt in the folder containing the source code and CSV file.

### Step 5: Compile the Program

Execute the compilation command shown below.

### Step 6: Execute the Program

Run the program using the default CSV file or provide a custom filename.

### Step 7: View the Output

The program displays the sales summary and generates report.txt.

```
ishikapuri@Ishikas-MacBook-Air C++ % clang++ -std=c++17 -Wall -o sales_summary sales_summary.cpp
./sales_summary
========== SALES SUMMARY ==========
Months analysed     : 4
Total revenue       : 53750.00
Total units sold    : 645
Average revenue     : 13437.50 per month
Average unit price  : 83.33

Best month          : April (17250.00)
Worst month         : March (9500.00)
Best vs worst gap   : 7750.00

Months above average: February April

Month-to-month growth:
  February      +25.00 %
  March         -36.67 %
  April         +81.58 %

Revenue chart:
January     | ########################## 12000
February    | ################################## 15000
March       | ###################### 9500
April       | ######################################## 17250
==================================
Report saved to report.txt
```

### Example CSV input

```
Month,Revenue,UnitsJanuary,45000,320February,38000,290March,52000,350
```

### Compile the program

```
g++ -std=c++17 -o sales main.cpp
```

### Run with the default input file

```
./sales
```

### Run with a different input file

```
./sales mydata.csv
```

After successful execution, the report is saved as report.txt in the current folder.

# 9. LIMITATIONS AND FUTURE SCOPE

## 9.1 Limitations

The program supports simple comma-separated CSV fields and does not handle commas inside quoted values.

The input records are expected to be arranged in chronological order.

Month-to-month growth depends on the order of records in the CSV file.

The average unit price calculation requires a non-zero total number of units sold.

The program generates a text-based bar chart instead of a graphical chart.

The current version focuses on monthly sales analysis and does not provide detailed product-wise or region-wise analysis.

## 9.2 Future Scope

Interactive Menu: Add a menu-driven interface to perform different operations.

Search Functionality: Allow users to search for sales information for a particular month.

Product-Wise Analysis: Include product details to compare the performance of different products.

Region-Wise Analysis: Add geographical information to analyse sales across different regions.

Graphical Visualization: Introduce graphical charts for better representation of sales data.

Quarterly and Yearly Reports: Add options to calculate quarterly and yearly sales summaries.

Improved Data Validation: Include stronger validation for negative values, duplicate months and missing columns.

Multiple Export Formats: Allow users to export reports in formats such as CSV and PDF.

# 10. CONCLUSION

The Sales Data Summary Tool successfully demonstrates the practical application of C++ programming in data processing and analysis.

The project automates the process of reading monthly sales records, calculating important statistics and generating an organised sales report. It helps identify sales trends, compare monthly performance and understand revenue growth.

Through this project, various C++ programming concepts such as structures, vectors, functions, file handling, strings, standard algorithms and exception handling have been implemented.

The application reduces manual calculations, saves time and presents sales information in an organised manner. Overall, this project provides a useful foundation for understanding how programming can be applied to real-world data analysis problems. It can be further enhanced by adding graphical visualisation, interactive